

Recursive Retraceability (A1.07): Entity-Level Retractable Chains at Cell, Aspect, and Self Scope in the CKS Multi-Level Architecture

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 12, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in “The Instinct/Reasoning Separation Outside the Model” (Li, April 2026), the second paper in the CKS theory series following “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems” (Li, April 2026).

It does not introduce new architectural commitments. Its contribution is to articulate, in operational form, how Paper 1’s commitment to path retraceability (A1.07) applies recursively across the three-level structure Paper 2 establishes — formalizing entity-level retractable chains at cell, aspect, and Self scope, and naming the nested structure through which those chains compose into deployment-wide audit capability.

Abstract

Paper 2 specifies through its recursive Paper 1 commitments treatment (B1.20) that all six Paper 1 architectural commitments apply at every level of the three-level structure — cell, aspect, and Self. This note formalizes the recursive application of commitment A1.07 (path retraceability and the accountability vocabulary). Recursive A1.07 does not mean that one deployment-level retractable chain covers all operations. It means that cell operations are independently retractable through the cell Action layer and cell lineage; aspect operations are independently retractable through aspect lineage and coordination event records; Self operations are independently retractable through Self lineage and integration event records. The retractable chains are structurally distinct at each level and nest: a complete Self-level operation is retractable through Self lineage into aspect contributions and from aspect contributions into cell contributions, with lineage anchors at each level tracing back to birth. The mechanism at every level is A2.40 provenance. The architectural result is entity-level audit capability that is both granular (cell-scope task instances are retractable individually) and complete (cross-level audit from Self down through aspects to individual cells is structurally available). This is the eighth note in the B1.20 decomposition series.

1. Why recursive retraceability requires standalone formalization

Path retraceability in Paper 1 is stated at cell scope: any substrate content can be traced back through a path of antecedent substrate content to the inputs and orchestration rules that produced it, with A2.40 provenance as the mechanism. The property is load-bearing at cell scope — it is what allows any piece

of substrate state to be interrogated after the fact for the authority chain under which it was produced, without consulting external logs, agent memory, or human recollection.

Paper 2 introduces a three-level architecture — cell, aspect, Self — and specifies through B1.20 that Paper 1's six commitments hold recursively at every level. For most commitments, the recursive application can be inferred without much risk of misreading. For A1.07, the recursive application requires careful formalization because the commitment has a specific structural shape — a chain from current state back to birth — and that shape must be instantiated independently at each level, with each level's chain having distinct content. Without formalization, two misreadings are available. The first is that A1.07 covers all levels via the cell-level chain alone: one reads any cell operation and thereby retraces everything above it. The second is that A1.07 collapses into one deployment-level chain that covers all operations across all levels in one flat record. Both misreadings lose the entity-level granularity that recursive A1.07 provides.

This note forecloses both misreadings by stating the recursive application precisely and naming what is structurally constant versus what differs across levels.

B2.105 is the eighth of thirteen notes decomposing B1.20. It follows B2.98 through B2.104, which have addressed six prior per-commitment recursions within the B1.20 series. B2.106 (recursive composition requirements, A1.13), B2.107 (recursive authority architecture), B2.108 (recursive conflict-as-first-class, A1.03), B2.109 (recursive operational tests), and B2.110 (recursive commitments verification) remain. The present note contributes the per-commitment formalization of level-specific retraceability.

2. The recursive application stated precisely

A1.07 at cell scope

At cell scope, A1.07 holds as Paper 1 specifies it, with the addition that Paper 2's lineage architecture names the anchor explicitly.

The cell lineage anchor (B2.43). The cell's birth record — the substrate entry created when the cell is born under human governance — is the terminal point of the retraceable chain. All subsequent cell events are substrate content that traces back, through provenance references, to this birth record. Without the birth record, the retraceable chain has no terminus and the retraceability property is structurally incomplete.

The cell Action layer (B2.26). The Action layer is the substrate layer that accumulates recorded task instances: what inputs the cell received, what instinct/reasoning it applied, what outputs it produced, under what DNA version. The Action layer is the operational retraceability record within the lineage. It records not what the cell could do (that is the DNA layer's role) but what it did.

The cell retraceable chain. For any cell operation, the chain runs: current operation → prior operations in the Action layer → the DNA version governing those operations (per A6.02 retroactivity,

DNA version history is itself retraceable) → birth record as the lineage anchor. The chain is substrate content at every step. What is retraceable at cell scope includes: what inputs any cell received at any point in its history, what instinct/reasoning it applied, what outputs it produced, under what cell DNA version, and at what point in the cell's lifecycle.

A1.07 at aspect scope

The aspect lineage anchor (B2.43). The aspect's birth record is the lineage anchor. All subsequent aspect coordination events chain back to it through provenance references.

Aspect coordination records. The aspect's coordination record layer holds: records of aspect invocations, the outcomes of cell coordination for the aspect's purpose, conflict-handling events among constituent cells, and membership change events (cells joining or leaving the aspect). These records are the operational retraceability record at aspect scope — they record what the aspect did as a coordination entity, not what any individual constituent cell did.

The aspect retraceable chain. For any aspect coordination operation, the chain runs: current coordination event → prior coordination events in the coordination record layer → the aspect DNA version governing that coordination → birth record as the lineage anchor. What is retraceable at aspect scope includes: which cells participated in which coordination, what coordination outcomes resulted, what conflicts were handled and how, and under what aspect DNA version.

A1.07 at Self scope

The Self lineage anchor (B2.43). The Self's birth record is the lineage anchor. All subsequent Self-level integration events chain back to it.

Self integration records. The Self's integration record layer holds: records of Self-level integration decisions, cross-aspect governance events, instinct/reasoning configuration changes, and structural reorganization decisions. These records capture the Self's operations as an integrated whole — not the operations of any particular aspect or cell within it.

The Self retraceable chain. For any Self integration operation, the chain runs: current integration event → prior integration events in the integration record layer → the Self DNA version governing that integration → birth record as the lineage anchor. What is retraceable at Self scope includes: what integration decisions the Self made, how cross-aspect governance was exercised, how the instinct/reasoning configuration evolved, and under what Self DNA version.

Nested retraceability

The three level-specific chains are structurally distinct but architecturally composable. A complete Self-level operation is retraceable not only through Self lineage but also, by following contributions downward, through the aspect contributions that composed it and the cell contributions that composed those aspects. The nesting structure is: Self lineage → aspect lineage entries for contributing aspects → cell Action layer entries for contributing cells → birth anchors at each level. Following the full nested

chain provides a complete account of how a Self-level operational result was produced from cell-level operations through aspect-level coordination to Self-level integration.

What is constant and what differs across levels

The retraceability principle is constant: at every level, every operation has a retraceable chain from the current state back to the entity's birth record. A2.40 provenance is the mechanism at every level — the six provenance metadata fields (writer attribution, timestamp, governing orchestration rule identifier, antecedent references, and the additional fields A2.40 specifies) are carried by every piece of substrate content at every level.

What differs is the content of the retraceable chain. Cell chains are task-instance granular: individual processing decisions, individual input/output records, and individual DNA-version transitions are the chain's nodes. Aspect chains are coordination-level: the nodes are coordination outcomes, conflict-handling events, and membership changes. Self chains are integration-level: the nodes are integration decisions and cross-aspect governance events. The granularity of what is retraceable is thus level-specific, and this is architecturally appropriate — the Self's retraceable chain does not need to contain every task-instance record from every constituent cell; those records are accessible through the nesting structure when needed.

3. What makes recursive retraceability architecturally distinctive

The contrast with conventional AI deployment architectures is instructive. Most deployed AI systems maintain operational logs at the deployment level: a deployment may record that a request was made, that a model was invoked, that a response was returned. In some deployments, logs include timing, cost, and error records. These deployment-level logs are useful for operational monitoring. What they do not provide is entity-level retraceable chains in the CKS sense.

An entity-level retraceable chain, as A1.07 recursive application defines it, requires three properties that deployment-level logs typically do not satisfy. First, the chain must be anchored to the entity's birth record, not to deployment initialization or session start. Second, the chain must carry provenance at every step — each node in the chain must record not only what happened but under what authority and governing rule. Third, the chain must be readable as substrate content, not as an external log that the system holds separately from its operational state.

Recursive A1.07 creates these entity-level chains at three levels of granularity simultaneously. The cell-level chain is the finest-grained: it enables audit of individual task instances, down to the level of what specific inputs a cell received and what instinct/reasoning it applied under what DNA version. This is a level of operational accountability that deployment-level logging does not provide and that agent-memory architectures actively undermine, because agent memory is not carried as substrate content and does not carry A2.40 provenance fields.

The nested structure provides the further property that cross-level audit is structurally available rather than requiring manual correlation across separate logs. An auditor who needs to understand why a Self-level integration decision was made can follow the Self retraceable chain; at the points where aspect contributions are referenced, they can follow aspect chains; at the points where cell contributions are referenced, they can follow cell chains. The audit path is substrate navigation, not external log correlation.

4. The biological analog

Biological organisms maintain genealogical traceability at multiple levels: individual cells within a multicellular organism can be traced to precursor cells through cell lineage; tissue lineages can be traced to germ layer origins; the organism's lineage can be traced through evolutionary history. At each level, the relevant retraceable chain is distinct in content — the cell lineage does not contain the tissue-level developmental history in the same record, and the tissue-level history does not contain the organism's evolutionary history in the same record. The chains are structurally distinct at each level, and tracing from one level to another requires moving between levels.

CKS recursive A1.07 instantiates this structure as a governed architectural commitment. The cell retraceable chain is the analog of cell lineage within the organism; the aspect chain is the analog of tissue-level developmental lineage; the Self chain is the analog of organism-level history. The key architectural difference is that CKS chains are explicit, complete, and substrate-readable — biological lineages are reconstructed from molecular evidence and are never complete. The mechanism that makes CKS chains explicit is A2.40 provenance, which biology has no equivalent for.

The biological analog is scaffolding for conceptual orientation; the architectural content is the A2.40-anchored entity-level retraceable chain structure described in §2.

5. Inherited Paper 1 commitments

A1.07 path retraceability is the commitment being formalized recursively. The recursive application extends the single-level path retraceability of Paper 1 to three structurally distinct chains, one at each CKS architectural level.

A2.40 provenance (six provenance metadata fields as substrate requirement) is the mechanism that makes each level's retraceable chain complete. Every substrate content entry at every level — cell Action layer entries, aspect coordination records, Self integration records — must carry the A2.40 provenance fields. Gaps in provenance are gaps in retraceability; complete provenance is what makes complete retraceable chains structurally available.

B2.43 birth lineage establishment provides the anchor for each level's retraceable chain. The birth record is the terminal substrate entry that all subsequent entries in the lineage trace back to. Recursive A1.07 depends on birth lineage establishment at every level — without birth records for cells, aspects, and Selves, the retraceable chains have no terminus.

B2.26 Action layer is the operational retraceability record at cell scope. The Action layer is where the content of cell-level retraceable chains accumulates: the recorded task instances that constitute the cell's operational history.

A6.02 retroactivity (DNA version history within retraceable chains) ensures that the DNA version governing any historical operation is itself retraceable. When a cell's DNA evolves across multiple versions, each historical operation is retraceable to the DNA version that governed it at the time of execution, not only to the current DNA version.

6. Operational implications

Entity-level chain maintenance. Deployments implementing recursive A1.07 maintain retraceable chains at each of the three levels independently. This means cell birth records, aspect birth records, and Self birth records are created at entity origination and preserved for the entity's lifetime and beyond. Action layer records, aspect coordination records, and Self integration records accumulate as operational history and carry A2.40 provenance at each entry.

Cell-level granularity as the finest-grained audit target. For deployments requiring the finest-grained audit, cell-level chains through the Action layer provide task-instance-resolution accountability. This is the most specific audit available in the architecture: what a particular cell did with a particular input under a particular DNA version at a particular point in its lifecycle.

Cross-level audit through nested retraceability. For deployments requiring deployment-wide audit — understanding how a Self-level outcome was produced from cell-level operations — the nested retraceability structure is the mechanism. An auditor follows the Self chain to aspect contributions, aspect chains to cell contributions, and cell chains to individual task instances, with each step being a substrate navigation rather than an external log query.

Death preserves closed chains. When a cell, aspect, or Self undergoes death per B1.11, the lineage chain is closed rather than deleted. The closed chain remains substrate-addressable for archived retraceability. Functional obsolescence death and lineage supersession death both preserve the closed chain; the distinction is in what triggers closure, not in whether the chain is preserved. This means that entities no longer operationally active remain retraceable — their history is available for audit regardless of their operational status.

Compliance demonstration at any level. Because chains are maintained independently at each level, compliance demonstration can target the appropriate level. Compliance questions about a specific cell's behavior are answered by the cell-level chain. Compliance questions about coordination-level behavior are answered by the aspect-level chain. Compliance questions about integration-level governance are answered by the Self-level chain. Cross-level compliance demonstration follows the nested structure.

7. Limits

Each level records its own scope. The cell Action layer records cell-level task instances; it does not record aspect-level coordination events or Self-level integration decisions. Aspect coordination records do not subsume cell-level task instance records. For complete audit, nested retraceability chains are followed — the levels do not collapse into one chain, and no single chain provides complete visibility across all levels independently.

Retraceability depends on complete A2.40 provenance. If substrate content at any level is written without complete A2.40 provenance fields, the retraceable chain has gaps at those points. Recursive A1.07 specifies that provenance must be complete at every level; it does not guarantee that any particular deployment has in fact maintained complete provenance. The architectural commitment is the requirement; operational compliance depends on implementation.

Archived entities remain retraceable; the chains nest but do not collapse. An archived cell's retraceable chain remains available as a closed chain. A Self's retraceable chain references aspect and cell contributions through nesting but does not contain those contributions' full chains internally — the nesting structure is followed by navigation, not by embedding all lower-level chains within the higher-level chain. The chains are structurally distinct at every level and remain so.

Retraceability is of the substrate path, not the LLM's internal reasoning. As A1.07 specifies at cell scope, path retraceability covers the substrate path — the chain of antecedent substrate content — not the LLM's internal reasoning process that produced any particular output. LLM outputs that affect substrate state are recorded in substrate with attribution per A2.23, but the LLM's inference is not retraceable. Recursive A1.07 extends this scope exclusion across all three levels.

8. One-sentence test

A deployment instantiates recursive A1.07 if and only if: for any cell, aspect, or Self operation, a reader can reconstruct the complete retraceable chain from that operation back to the entity's birth record by reading substrate content alone at the appropriate level, using A2.40 provenance fields at each step, without consulting external logs, agent memory, or human recollection.

9. Why naming as standalone matters

Formalizing recursive A1.07 as a standalone note serves the same purpose as formalizing each other per-commitment recursion in the B1.20 series: it creates public prior art at the level of the specific recursive application, not only at the level of the general recursive-levels principle. A party who wished to claim novelty in entity-level retraceable chains within multi-level AI substrate architectures — the cell Action layer as the operational record within a cell lineage, or aspect coordination records as the retraceability substrate for multi-cell coordination, or the nested structure through which Self-level operations are auditable through aspect and cell chains — would need to engage this formalization as prior art.

The note also articulates the precise relationship between the three level-specific chains. That relationship — constant principle and mechanism, distinct chain content per level, composable nesting — is itself patentable territory as an architectural specification, independent of the per-level chains considered individually.

B2.105 is the eighth of thirteen notes in the B1.20 decomposition. The series progress is: B2.98 through B2.104 have addressed seven prior per-commitment recursions; B2.105 addresses recursive retraceability A1.07; B2.106 through B2.110 will address recursive composition requirements, recursive authority architecture, recursive conflict-as-first-class, recursive operational tests, and recursive commitments verification, closing Phase B2 of the derivation note series.

Source papers

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Recursive Retraceability (A1.07): Entity-Level Retraceable Chains at Cell, Aspect, and Self Scope in the CKS Multi-Level Architecture*. May 12, 2026. ORCID: 0009-0004-8065-3235.